EE6094
CAD for VLSI Design

PA3
Static Timing Analysis

Andy, Yu-Guang Chen
Assistant Professor, Department of EE
National Central University
andyygchen@ee.ncu.edu.tw

Outline

- Problem description
- Input format
- Output format
- Evaluation
- Requirement

Outline

- Problem description
- Input format
- Output format
- Evaluation
- Requirement

Problem description

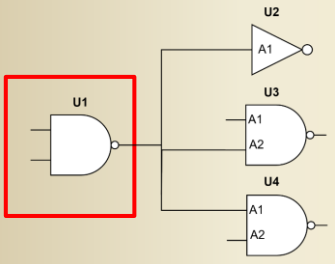
- Input
  - A gate-level flatten netlist
  - A liberty file
- Output
  - Output load
  - Delay
  - Path

Problem description

- Determine output load of each gate.
- Determine delay of each gate.
- Find the longest path and the shortest path.

Problem description

- Calculate output load
  - The summation of the input capacitance of all the fanout cells.

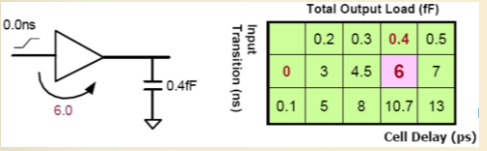


✓ Output load(U1)

= input cap.(U2/A1) +
input cap.(U3/A2) +
input cap.(U4/A1)

Problem description

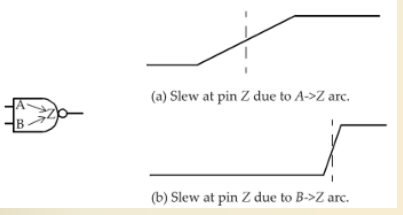
- Delay Calculation on a Cell
  - Lookup table
    - Input transition time (slew)
    - Output capacitance
    - Delay (rise/fall time)
    - Output transition time (slew)



	0.2	0.3	0.4	0.5
0	3	4.5	6	7
0.1	5	8	10.7	13

Problem description

- Delay Calculation on Multi-input Cell
  - Multiple timing arcs (delay and slew) may be found, input transition time is defined by the input that arrive latest.



Problem description

- Worst-case Output Calculation
  - Calculate the propagation delay of both output situations (output = 0 and output = 1).
  - The one that has larger propagation delay will be the worst-case output, then output transition time is defined according to worst-case output.
  - Propagation delay ➔ rise/fall time
  - Output transition time ➔ rise/fall transition time

Problem description

- Find longest/ shortest path
  - According to the delay information, compare the worst-case delay of all outputs.
  - The largest one is the longest delay, and the smallest one is the shortest delay.

Problem description

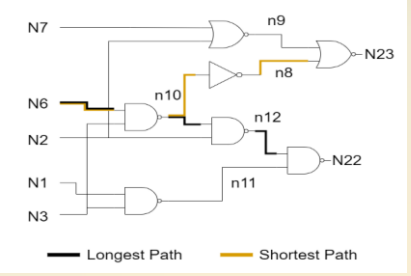
- Boundary Condition & Assumption
  - Input transition time set to 0ns
  - Output loading set to 0.03pF
  - Wire delay set to 0.005ns

Outline

- Problem description
- Input format
- Output format
- Evaluation
- Requirement

Input format

```
module c17 (N1, N2, N3, N6, N7, N22, N23);
    input N1, N2, N3, N6, N7;
    output N22, N23;
    //internal wires
    wire n8, n9, n10, n11, n12;
    /*
        */
    NOR2X1 U8 (.A1(n8), .A2(n9), .ZN(N23));
    NOR2X1 U9 (.A1(N2), .A2(N7), .ZN(n9));
    INVX1 U10 (.I(n10), .ZN(n8)); //invertor
    NANDX1 U11 (.A1(n11), .A2(n12), .ZN(N22));
    NANDX1 U12 (.A1(N2), .A2(n10), .ZN(n12));
    NANDX1 U13 (.A1(N6), .A2(N3), .ZN(n10));
    NANDX1 U14 (.A1(N1), .A2(N3), .ZN(n11));
endmodule
```



The gate-level netlist represent the whole circuit diagram.

Input format

- .LIB (Liberty File) Format
  - Unit attributes
  - Look-up table template
  - Gate information
    - I/O pin information
    - rise/ fall power
    - rise/ fall time
    - rise/ fall transition time

Input format

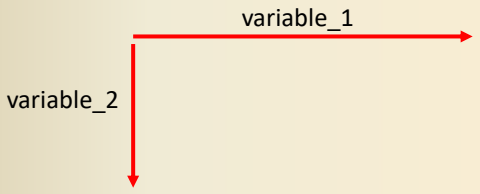
- .LIB (Liberty File) Format
  - Unit attributes

```
/* unit attributes */
  time_unit : "1ns";
  voltage_unit : "1V";
  current_unit : "1mA";
  capacitive_load_unit (1.0,pf);
```


Input format

- .LIB (Liberty File) Format
  - Unit attributes
  - Look-up table template

```
lu_table_template(table10){
  variable_1 : total_output_net_capacitance;
  variable_2 : input_transition_time;
  index_1 ("0.001400,0.003000,0.006200,0.012500,0.025100,0.050400,0.101000");
  index_2 ("0.0208,0.0336,0.06,0.1112,0.2136,0.4192,0.8304");
}
```



Input format

- .LIB (Liberty File) Format
  - Unit attributes
  - Look-up table template
  - Gate information

```
cell (NOR2X1) {
  pin(A1) {
    direction : input;
    capacitance : 0.0105008;
  }
  pin(A2) {
    direction : input;
    capacitance : 0.0108106;
  }
  pin(ZN) {
    direction : output;
    capacitance : 0.0;
```

```
timing() {
  cell_rise(table10){
    values ("0.023104,0.026347,0.032439,0.037211,0.050187,0.075629,0.10893",\
            "0.027062,0.029858,0.036397,0.042338,0.055823,0.083858,0.11942",\
            "0.034333,0.037002,0.043649,0.050318,0.064815,0.089926,0.13827",\
            "0.048838,0.051443,0.053267,0.064115,0.083493,0.11205,0.17069",\
            "0.076172,0.077668,0.081485,0.08978,0.11263,0.14904,0.20359",\
            "0.13351,0.13486,0.13847,0.14635,0.16278,0.20974,0.27949",\
            "0.24777,0.24888,0.25234,0.25971,0.27596,0.30968,0.40393");
            }
  cell_fall(table10){
    values ("0.017693,0.020473,0.024855,0.029528,0.03475,0.038561,0.030505",\
            "0.019856,0.022665,0.027792,0.033415,0.03982,0.045554,0.039165",\
            "0.024076,0.026936,0.032932,0.040339,0.048928,0.057779,0.054661",\
            "0.03228,0.035141,0.041568,0.051796,0.064313,0.069003,0.080192",\
            "0.048376,0.051227,0.057825,0.070186,0.089026,0.09852,0.11886",\
            "0.078389,0.08013,0.090011,0.091689,0.11283,0.15092,0.17976",\
            "0.14202,0.14372,0.14735,0.15483,0.17074,0.21341,0.2901");
            }
```


Outline

- Problem description
- Input format
- Output format
- Evaluation
- Requirement

Output format

◆Load_StudID_Casename.txt

```
U8  0.030000
U11 0.030000
U13 0.018896
U9  0.010811
U10 0.010501
U12 0.007985
U14 0.006836
```

U8, U11 ➔ Same loading ➔ Sort instance number in ascending order

Output format

◆Delay_StudID_Casename.txt

- Each row shows an instance name, worst-case output value, corresponding worst-case delay, and output transition time in order.

```
U11 1 0.070545 0.116993
U8 1 0.064212 0.103237
U12 1 0.044436 0.059305
U10 0 0.035620 0.042872
U13 1 0.033812 0.054006
U9 1 0.028109 0.053694
U14 1 0.024308 0.025355
```

Output format

- Path_StudID_Casename.txt
  - Output the longest and the shortest delays and their corresponding delay path.
  - If the instance has multiple input signals that has same arrival time, you can output either one of them.

```
Longest delay = 0.158793, the path is: N6 -> n10 -> n12 -> N22
Shortest delay = 0.143645, the path is: N6 -> n10 -> n8 -> N23
```


Outline

- Problem description
- Input format
- Output format
- Evaluation
- Requirement

Evaluation

- Correctness of your code: 40%
  - Step1: 15%
  - Step2: 15%
  - Step3: 10%
- Performance: 20%
- Readability of your code: 10%
- The report: 10%
- Demo session: 20%

Correctness

- We will have 2 public cases and 3 hidden cases to judge your program
- TA will test your program:
  - ./$(TARGET) input.v test_lib.lib
- You will get the point if you correctly implemented the algorithm and pass the checker for each cases

Performance

- We will rank the runtime for each cases to judge the performance of your PA3
- #include<ctime> or #include<chrono>

```
#include "STA.h"
#include <ctime>

int main(int argc, char* argv[]){
    clock_t start = clock();
    STA sta;
    sta.parser_gatelevel_netlist(argv[1]);
    sta.parser_liberty_file(argv[2]);
    sta.ouput_load();
    sta.output_delay();
    sta.output_path();
    clock_t end = clock();
    double duration = (double)(end - start) / CLOCKS_PER_SEC;
    cout << "Execution time: " << duration << "s" << endl;
    return 0;
}
```

```
#include "STA.h"
#include <chrono>

int main(int argc, char* argv[]){
    auto start = chrono::high_resolution_clock::now();
    STA sta;
    sta.parser_gatelevel_netlist(argv[1]);
    sta.parser_liberty_file(argv[2]);
    sta.ouput_load();
    sta.output_delay();
    sta.output_path();
    auto end = chrono::high_resolution_clock::now();
    auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
    cout << "Execution time: " << duration.count() << " microseconds" << endl;
    return 0;
}
```


Readability

- Function cuts
- The name of the variables
- Typesetting

The report

- In your report, you have to include at least:
  - Compilation and execution
  - Completion
  - Method of your design
  - Test cases results
  - Hardness
  - Suggestions
- We don’t restrict the report format and length

The report

- You can also include
  - Cover
  - Flow charts
  - Code explanations
    - Concepts, functions, variables, etc.
- The grading of the report will compare yours with others

Outline

- Problem description
- Input format
- Output format
- Evaluation
- Requirement

Submission

- You must submit a single .tar file named StudID_PA3.tar
- The directory structure inside should follow this format; **otherwise, your program will not be able to run correctly and penalties will apply.**

```
9862534_PA3/                  (Project folder)
├── 9862534_PA3.cpp           (Main source file)
├── 9862534_PA3_report.pdf    (Report file - only .pdf is accepted)
├── inc/                      (Header files, if needed)
│   ├── file1.h
│   ├── file2.h
├── src/                      (Source files, if needed)
│   ├── file1.cpp
│   ├── file2.cpp
```

Demo session

- You must show up in the demo session; **otherwise, you will get 0 point of this PA**
- Be familiar with your code

Q&A

Email: dinghc99@gmail.com

Andy, Yu-Guang Chen
Assistant Professor, Department of EE, NCU
Email: andyygchen@ee.ncu.edu.tw
FB: Yu-Guang Chen
IG: ncu.eda.andy
Google account: andyygchen.ncu